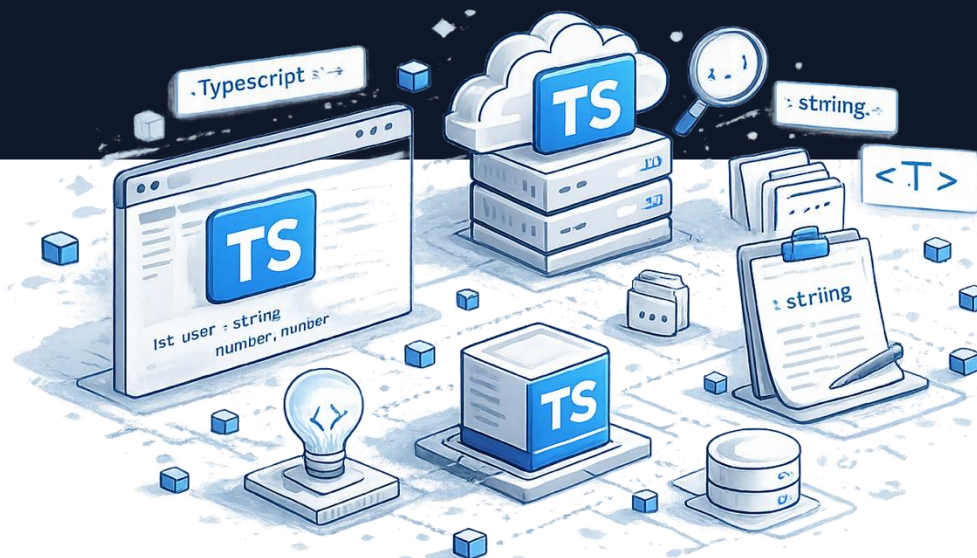




Progettazione di applicazioni web full stack

Laboratorio TypeScript



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia



Laboratorio TypeScript in Nx

- Come sviluppare package TypeScript in un monorepo
- Come rendere lo sviluppo più efficiente con il meccanismo di caching di Nx
- Come eseguire e testare il codice sviluppato su più package
- Come condividere codice tra package utilizzando librerie locali

Perché TypeScript in un monorepo



- Problema: JS «ti lascia fare tutto», Nx monorepo amplifica i rischi (più team, più **libs**)
- TS come «*contratto*»: riduce regressioni e rende il refactoring più sicuro
- In Nx: TS è la colla tra apps e libs (import tramite alias)



TypeScript Configuration Check

Come capire se nel monorepo TypeScript è configurato correttamente?

- Esiste un file `tsconfig.base.json` e/o `tsconfig.json` nella root del workspace

- Nel file `package.json` si vedono dipendenze del tipo `typescript`, `@nx/js` e/o `@nx/node`, `@nx/nest`
- Che tramite il comando `npx nx list` si vedano plugin come `@nx/js`, `@nx/node`

- Tre comandi che devono funzionare:

```
npx tsc -v  
npx nx build <progetto>  
npx nx test <progetto>
```

Creare una nuova lib in TypeScript



- Comando di base **npx nx g @nx/js:lib <nome libreria>**
- Dopo la generazione, si possono eseguire i diversi comandi **lint**, **test** e **build**
- La compilazione crea una cartella **dist** con il codice Javascript pronto per l'esecuzione

```
npx nx g @nx/js:lib libs/hello-ts
```

```
npx nx lint hello-ts
```

```
npx nx test hello-ts
```

```
npx nx build hello-ts
```

Il processo di compilazione



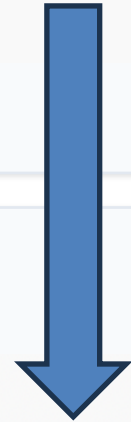
- TypeScript non viene eseguito direttamente
- A *runtime gira Javascript*
- Il compilatore TypeScript (**tsc**) prende i file **.ts** e produce file **.js**
- Cosa succede durante la compilazione?
 - **Type-check**: controlli la correttezza dei tipi (errori «*prima di eseguire*»)
 - **Rimozione dei tipi**: annotazioni tipo **:numer**, **interface**, **type** «spariscono», perché non esistono in Javascript
 - **Trasformazioni di sintassi**: se necessario, per adeguarsi ad una versione JS più compatibile (dipende da **target** nel file **tsconfig**)

Il processo di compilazione: esempio



```
function add(a: number, b: number): number {  
    return a + b;  
}
```

TypeScript



Javascript

```
function add(a, b) {  
    return a + b;  
}
```

Strategie di compilazione differenti



- Alcune opzioni di compilazione (`npx nx g @nx/js:lib --help`)
 - `--bundler` = come Nx *esegue la build/pacchettizza*
 - Scelte: `tsc`, `swc`, ... `none` (la scelta `none` implica che la libreria non è «*buildable*»)
 - `--compiler` = che compilatore viene usato (`tsc` o `swc`)

```
npx nx g @nx/js:lib libs/hello-tsc --bundler=tsc -compiler=swc
```

- **TSC**: compila TS → JS e fa type-check approfondito
 - suggerito nelle prime fasi di sviluppo
- **SWC**: compila molto velocemente
 - Per velocizzare il lavoro nel caso di compilazioni frequenti

Come capire che compilatore stiamo usando?



- Tramite il comando:

```
npx nx show project <project>
```

- Nella sezione **Target.build** trovo evidenza del compilatore utilizzato

Targets

^ build **ts**

Cacheable



Created by @nx/js/typescript from libs/hello-tsc/tsconfig.lib.json

nx run @org/hello-tsc:build

Description

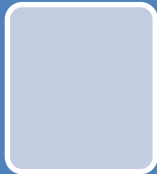
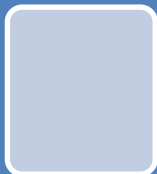
Builds the project with `tsc`.

Command

Strategie diverse nello stesso progetto



Installiamo le dipendenze SWC
(solo la prima volta)



```
npx nx add @swc/core
```

```
npx nx add @swc/cli
```

```
cp {projectRoot}/.spec.swcrc  
{projectRoot}/.swcrc
```

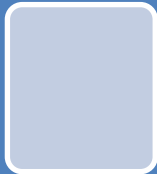
Strategie diverse nello stesso progetto



Installiamo le dipendenze SWC
(solo la prima volta)



Manteniamo TSC come base
(rigorosa)



Targets

^ build TS

Created by @nx/js/typescript from libs/hello-tsc/tsconfig.lib.json

Description

Builds the project with `tsc`.

Command

`tsc --build tsconfig.lib.json` [🔗](#)

Strategie diverse nello stesso progetto



Installiamo le dipendenze SWC
(solo la prima volta)



Manteniamo TSC come base
(rigorosa)



Aggiungiamo un nuovo target
nel file di progetto
package.json



```
"nx": {  
  "targets": {  
    "build-swc": {  
      "executor": "@nx/js:swc",  
      "outputs": ["{projectRoot}/dist"],  
      "options": {  
        "main": "{projectRoot}/src/index.ts",  
        "outputPath": "{projectRoot}/dist",  
        "tsConfig": "{projectRoot}/tsconfig.lib.json"  
      }  
    }  
  }  
}
```

File **package.json**

Strategie diverse nello stesso progetto



Installiamo le dipendenze SWC
(solo la prima volta)



Manteniamo TSC come base
(rigorosa)



Aggiungiamo un nuovo target
nel file di progetto
package.json



D'ora in poi, disponiamo di
comandi distinti per le diverse
strategie di compilazione

```
npx nx build {projectName}
```

```
npx nx run {projectName}:build-swc
```



Esempio: visualizzatore ordini

TypeScript + Nx (libs + test)

Cosa faremo

- Una pagina web con un bottone che:
 - *simula la ricerca di un ordine* in un catalogo/archivio
 - mostra *successo/errore* in modo tipizzato
 - Stampa un riepilogo del metodo di pagamento

1. Legge (o simula) un **id** ordine
2. Lo valida (**ItemIdParser**) → **Result**
3. Cerca l'ordine in un catalogo (**InMemoryOrderRepository**)
4. Se lo trova, mostra importo e metodo di pagamento (*union type* **CART** | **PAYPAL**)
5. Se non lo trova, mostra un messaggio **NOT_FOUND**



Esempio: visualizzatore ordini

TypeScript + Nx (libs + test)

1. *Separazione delle responsabilità* → la logica starà in una libreria riutilizzabile (**libs/hello-tsc**), la UI sta in una web app (**apps/static**)
2. *Errori e stati nel tipo* → invece di **throw**, verrà usato **Result<T,E>**: la gestione degli errori diventa parte del flusso
3. *Union types + narrowing* → **PaymentMethod** è **CARD | PAYPAL**; TS guida e non permette di accedere a campi sbagliati
4. *Interfacce e classi* → **Parser<T>** e **Repository<T>** definiscono contratti; le classi sono implementazioni concrete (e sostituibili)

Generazione della libreria in Nx



```
npx nx g @nx/js:lib libs/hello-ts --bundler=tsc --unitTestRunner=jest --  
linter=eslint
```

- Nx crea i target **build/test/lint**
- Uso di **jest** come framework di testing
- Uso di **eslint** per il controllo statico del codice

Verifica: i target test esistono

```
npx nx build hello-ts  
  
npx nx lint hello-ts  
  
npx nx test hello-ts
```




1) Definiamo i tipi (contratti) – File `src/lib/hello-ts.ts`

- Type union per `Result` (discriminata dalla proprietà `ok`)
- Type union per `PaymentMethod` (discriminata dalla proprietà `kind`)
- Type alias per `ItemId`
- Tipo complesso (modello) per `Order`

```
// 1) Definizione dei tipi
// Union type discriminata: Result<T,E> (success/failure)
export type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };

// Union type discriminata: "PaymentMethod"
export type PaymentMethod =
  | { kind: 'CARD'; last4: string }
  | { kind: 'PAYPAL'; email: string };

// Un tipo dominio
export type ItemId = string;

export type Order = {
  id: ItemId;
  amount: number;
  payment: PaymentMethod;
};
```



1) Definiamo i tipi (contratti) – File `src/lib/hello-ts.ts`

- Type union per **Result** (discriminata dalla proprietà **ok**)
- Type union per **PaymentMethod** (discriminata dalla proprietà **kind**)
- Type alias per **ItemId**
- Tipo complesso (model) **Order**

Type **Result**, unione di due forme possibili:

- caso successo: oggetto con **ok: true** e **value** di tipo **T**
- caso errore: oggetto con **ok: false** e **error** di tipo **E**

```
// 1) Definizione dei tipi
// Union type discriminata: Result<T,E> (success/failure)
export type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };

// Union type discriminata: "PaymentMethod"
export type PaymentMethod =
  | { kind: 'CARD'; last4: string }
  | { kind: 'PAYPAL'; email: string };

// Un tipo dominio
export type ItemId = string;

export type Order = {
  id: ItemId;
  amount: number;
  payment: PaymentMethod;
};
```



2) Sfruttiamo il meccanismo dell'ereditarietà tra tipi – File `src/lib/hello-ts.ts`

- Classe astratta `DomainError` che estende la classe *built-in* `Error`

```
throw new Error('Qualcosa è andato storto');
```

- Due classi (`InvalidInputError` e `NotFoundError`) che estendono la classe astratta

```
export abstract class DomainError extends Error {
  abstract kind: 'INVALID_INPUT' | 'NOT_FOUND';

  protected constructor(message: string) {
    super(message);
    this.name = this.constructor.name;
  }
}

export class InvalidInputError extends DomainError {
  kind = 'INVALID_INPUT' as const;

  constructor(public reason: string) {
    super(`Invalid input: ${reason}`);
  }
}

export class NotFoundError extends DomainError {
  kind = 'NOT_FOUND' as const;

  constructor(public id: string) {
    super(`Not found: ${id}`);
  }
}
```



2) Sfruttiamo il meccanismo dell'ereditarietà tra tipi – File `src/lib/hello-ts.ts`

- Classe astratta **DomainError** che estende la classe *built-in* **Error**

```
throw new Error('Qualcosa è andato storto');
```

- Due classi (**InvalidInputError**, **NotFoundError**)

Metodo **protected**:

- consentito: `new InvalidInputError()`
- non consentito: `new DomainError()`

```
export abstract class DomainError extends Error {
  abstract kind: 'INVALID_INPUT' | 'NOT_FOUND';

  protected constructor(message: string) {
    super(message);
    this.name = this.constructor.name;
  }
}

export class InvalidInputError extends DomainError {
  kind = 'INVALID_INPUT' as const;

  constructor(public reason: string) {
    super(`Invalid input: ${reason}`);
  }
}

export class NotFoundError extends DomainError {
  kind = 'NOT_FOUND' as const;

  constructor(public id: string) {
    super(`Not found: ${id}`);
  }
}
```



3) Interfaccia `Parser` e sua implementazione – File `src/lib/hello-ts.ts`

Definisce la funzione `parse`:

- prende input che deve essere una stringa
- Ritorna un successo con la stringa, oppure un errore tra quelli di `DomainError`

```
// 3) Interfaccia: contratto di un Parser
export interface Parser<T> {
  parse(input: string): Result<T, DomainError>;
}

// Classe che implementa un'interfaccia (Parser<ItemId>)
export class ItemIdParser implements Parser<ItemId> {
  parse(input: string): Result<ItemId, DomainError> {
    const id = input.trim();
    if (!id) return { ok: false, error: new InvalidInputError('Id vuoto') };
    return { ok: true, value: id };
  }
}
```



4) Interfaccia `Repository` e sua implementazione – File `src/lib/hello-ts.ts`

Definisce la funzione `findById`:

- prende un `id` in input
- se lo trova tra gli `orders`,
ritorna `ok: true`
(discriminante) e l'`id` come
valore
- altrimenti, ritorna `ok: false`
e un errore di tipo
`NotFoundError`

```
export interface Repository<T> {  
  findById(id: ItemId): Result<T, DomainError>;  
}  
  
export class InMemoryOrderRepository implements Repository<Order> {  
  private orders: Order[];  
  constructor(orders: Order[]) {  
    this.orders = orders;  
  }  
  
  findById(id: ItemId): Result<Order, DomainError> {  
    const found = this.orders.find((o) => o.id === id);  
    if (!found) return { ok: false, error: new NotFoundError(id) };  
    return { ok: true, value: found };  
  }  
}
```



5) Narrowing su type union – File `src/lib/hello-ts.ts`

Due funzioni per ritornare dei messaggi che sfruttano l'elemento discriminante dei type union:

- La proprietà `kind` di `PaymentMethod` per ritornare un messaggio sul metodo di pagamento
- La proprietà `ok` di `Result` per ritornare un messaggio sul tipo di errore

```
export function describePayment(p: PaymentMethod): string {
  switch (p.kind) {
    case 'CARD':
      return `Card **** ${p.last4}`;
    case 'PAYPAL':
      return `PayPal (${p.email})`;
  }
}

export function messageFor<T>(res: Result<T, DomainError>): string {
  if (res.ok) return `✅ OK: ${String(res.value)}`;

  // narrowing su error.kind
  switch (res.error.kind) {
    case 'INVALID_INPUT':
      return `❌ INVALID_INPUT: ${res.error as InvalidInputError}.reason`;
    case 'NOT_FOUND':
      return `❌ NOT_FOUND: ${res.error as NotFoundError}.id`;
  }
}
```



5) Public API della libreria – File `src/index.ts`

- Export puliti
- Chi consuma non importa da path profondi

```
export * from './lib/hello-ts.js';
```


Generazione della web app



```
npx nx add @nx/web
```

```
npx nx g @nx/web:app apps/static
```

- Nx crea una *web app* (HTML + CSS + TS)
- La web app sarà contenuta nella cartella *apps/static*

```
<!doctype html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>TS in Nx - Web</title>
  </head>
  <body>
    <h1>Demo</h1>

    <button id="run">Run demo</button>
    <pre id="out"></pre>

    <script type="module" src="/src/main.ts"></script>
  </body>
</html>
```

La web app fa uso delle librerie TS



6) Web app statica - File `src/main.ts`

```
import {
  ItemIdParser,
  InMemoryOrderRepository,
  describePayment,
  messageFor,
} from '@org/hello-ts';

const out = document.getElementById('out')!;
const btn = document.getElementById('run')!;

btn.addEventListener('click', () => {
  const parser = new ItemIdParser();

  const idRes = parser.parse(' o2 ');
  const lines: string[] = [];
```

La web app fa uso delle librerie TS



6) Web app statica - File `src/main.ts`

```
npx nx run static:serve
```

```
lines.push('parseId: ' + messageFor(idRes));

if (idRes.ok) {
  const repo = new InMemoryOrderRepository([
    { id: 'o1', amount: 100, payment: { kind: 'CARD', last4: '1234' } },
    { id: 'o2', amount: 50, payment: { kind: 'PAYPAL', email: 'a@b.com' } },
  ]);

  const orderRes = repo.findById(idRes.value);
  lines.push('findById: ' + messageFor(orderRes));

  if (orderRes.ok) {
    lines.push('payment: ' + describePayment(orderRes.value.payment));
  }
}

out.textContent = lines.join('\n');
});
```

Configurazione dell'ambiente TS



7) Web app statica - File `tsconfig.app.json`

- In quale ambiente girerà il codice TS?
 - Nel **browser** → viene caricato il **DOM**
 - In **Node** → non viene caricato il **DOM**, quindi non esiste l'oggetto **document**
- TS usa «cataloghi» di funzioni disponibili (**lib**) → in questo caso, la libreria **DOM**
- Questa operazione è valida solo perché stiamo considerando la web app, che gira nel browser

```
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "outDir": "dist",
    "types": ["node"],
    "rootDir": "src",
    "module": "esnext",
    "moduleResolution": "bundler",
    "tsBuildInfoFile": "dist/tsconfig.app.tsbuildinfo",
    "lib": ["DOM"]
  },
  "exclude": [
    "out-tsc",
    "dist",
    "src/**/*.spec.ts",
    "src/**/*.test.ts",
    "jest.config.ts",
    "jest.config.cts",
    "eslint.config.js",
    "eslint.config.cjs",
    "eslint.config.mjs"
  ],
  "include": ["src/**/*.ts"],
  "references": [
    {
      "path": "../../libs/hello-ts/tsconfig.lib.json"
    }
  ]
}
```

Rimuovere

"types": ["node"]

Aggiungere

"lib": ["DOM"]



8) Uso del framework di testing jest → `src/lib/hello-ts.spec.ts`

- Il framework **jest** offre una serie di API per definire i test:
 - **describe** – per creare un gruppo di test (una suite)
 - **test** – per definire un singolo caso di test
 - **expect().toBe()** – per definire asserzioni («mi aspetto che...»)

```
import {
  ItemIdParser,
  InMemoryOrderRepository,
  describePayment,
  messageFor,
} from './hello-ts.js';

describe('TS advanced example', () => {
  test('ItemIdParser parses valid id', () => {
    const parser = new ItemIdParser();
    const res = parser.parse(' o1 ');

    console.log(messageFor(res)); // messaggio in console

    expect(res.ok).toBe(true);
    if (res.ok) {
      expect(res.value).toBe('o1');
    }
  });
});
```



Progettazione di applicazioni web full stack Laboratorio TypeScript



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia